Cairo University

Faculty of Engineering

Advanced Tic Tac Toe Game

Testing file

Supervisor: Dr.Omar Nasr

ID	Name
9220822	مريم سامي راغب عطا
9230707	ماركو شنوده ثابت اسعد
9230373	رنا يوسف عبدالله صالح
9230246	اميرة سامح عبدالحليم سيد
9230384	ريم حسين احمد بسيوني

Table of Contents

1. Introduction

The testing documentation for the Advanced Tic Tac Toe Game offers an in-depth review of the testing efforts aimed at ensuring the game's functionality, reliability, and performance. This section details the purpose of the testing documentation, the target audience, and recommendations for understanding the testing process more effectively.

1.1 Purpose

The objectives of this testing documentation are to:

- Present a comprehensive account of the testing strategies, test cases, and coverage reports conducted for the Advanced Tic Tac Toe Game.
- Confirm the accuracy, dependability, and performance of the game's features through thorough testing.
- Serve as a reference for developers, testers, and project stakeholders to evaluate the game's quality and readiness for release.

2. Testing Strategies

2.1 Unit Testing

Unit testing is a critical strategy used in the testing documentation for the Advanced Tic Tac Toe Game. It involves testing individual units or components of the software in isolation to ensure their correctness and functionality. The primary focus is on validating key components such as the UserManager, AI algorithms, and other essential parts. All these tests are conducted within specific test files like usermanager_test.cpp, ai_test.cpp, medium_ai_test.cpp, hard_ai_test.cpp, and minimax_test.cpp.

The unit tests are designed to cover various aspects of the game, including:

1. **UserManager Tests (usermanager_test.cpp)**:
    - **Tested Functions**: registerUser(), loginUser().
    - **Covered Scenarios**: Successfully registering a new user, preventing duplicate usernames, logging in with valid credentials, and blocking login with invalid credentials.

```
#include <gtest/gtest.h>
#include "UserManager.h"
TEST(UserManagerTest, RegisterSuccess) {

EXPECT_TRUE(UserManager::registerUser("test_user",
"test123"));
}

TEST(UserManagerTest, RegisterDuplicate) {
    UserManager::registerUser("duplicate", "pass1");

EXPECT_FALSE(UserManager::registerUser("duplicate",
"pass2"));
}

TEST(UserManagerTest, LoginSuccess) {
    UserManager::registerUser("login_user",
"pass123");
    EXPECT_TRUE(UserManager::loginUser("login_user",
"pass123"));
}
```

2. **Easy AI Tests (ai_test.cpp)**:
   - **Tested Function**: easyAI().
   - **Covered Scenarios**: Chooses an available cell randomly and does not pick a cell already used.

```
#include <gtest/gtest.h>
#include <vector>
#include <set>
#include "../include/ai_logic.h"
#include <algorithm>
TEST(AITest, EasyAIChoosesValidMove) {
    std::vector<char> board =
{'1','2','3','4','5','6','7','8','9'};
    std::vector<int> used = {1, 2, 3};
    int choice = 0;

    AILogic::easyAI(board, used, choice);

    EXPECT_GE(choice, 1);
    EXPECT_LE(choice, 9);
    EXPECT_EQ(std::count(used.begin(), used.end(),
choice), 0);
```

3. **Medium AI Tests (medium_ai_test.cpp)**:
   - **Tested Function**: mediumAI().
   - **Covered Scenarios**: Blocks the player if they are about to win and makes smart decisions based on board state.

```
#include <gtest/gtest.h>
#include <vector>
#include "../include/ai_logic.h"
#include "../include/game_logic.h"
#include <algorithm>

TEST(MediumAITest, BlocksOpponentWin) {
    std::vector<char> board = {'X', 'X', '3', '4', 'O', '6', '7', '8', '9'};
    std::vector<int> used = {1, 2, 5};
    int choice = 0;
    AILogic::mediumAI(board, used, choice, 'O', 'X');
    // AI لازم تسد خانة 3
    EXPECT_EQ(choice, 3);
}

TEST(MediumAITest, TakesWinningMove) {
    std::vector<char> board = {'O', 'O', '3', '4', 'X', '6', '7', '8', '9'};
    std::vector<int> used = {1, 2, 5};
    int choice = 0;

    AILogic::mediumAI(board, used, choice, 'O', 'X');

    // AI لازم تاخد خانة 3 وتكسب
    EXPECT_EQ(choice, 3);
}
```

4. **Hard AI Tests (hard_ai_test.cpp)**:
   - **Tested Function**: hardAI().
   - **Covered Scenarios**: Uses Minimax algorithm and picks the optimal move for every situation.

```
#include <gtest/gtest.h>
#include <vector>
#include "../include/ai_logic.h"
#include "../include/game_logic.h"
#include <algorithm>

TEST(HardAITest, TakesWinningMove) {
    std::vector<char> board = {'O', 'O', '3', '4', 'X', '6', '7',
'8', '9'};
    std::vector<int> used = {1, 2, 5};
    int choice = 0;
    AILogic::hardAI(board, used, choice, 'O');
    EXPECT_EQ(choice, 3);  // O can win by playing at position 3
}

TEST(HardAITest, BlocksPlayerWinningMove) {
    std::vector<char> board = {'X', 'X', '3', '4', 'O', '6', '7',
'8', '9'};
    std::vector<int> used = {1, 2, 5};
    int choice = 0;

    AILogic::hardAI(board, used, choice, 'O');

    EXPECT_EQ(choice, 3);  // O must block X at position 3
}
```

5. **Minimax Algorithm Tests (minimax_test.cpp)**:
   - **Tested Function**: minimax() (made public for testing).
   - **Covered Scenarios**: Returns best score when AI can win, returns worst score when opponent can win, and evaluates board state correctly.

```
#include <gtest/gtest.h>
#include <vector>
#include "../include/ai_logic.h"
#include "../include/game_logic.h"
#include <algorithm>

TEST(MinimaxTest, DetectsWinForAI) {
    std::vector<char> board = {'O', 'O', '3', '4', 'X', '6', '7', '8', '9'};
    int score = AILogic::minimax(board, true, 'O', 'X');
    EXPECT_GT(score, 0); // AI (O) should detect winning move
}

TEST(MinimaxTest, DetectsLossForAI) {
    std::vector<char> board = {'X', 'X', '3', '4', 'O', '6', '7', '8', '9'};
    int score = AILogic::minimax(board, false, 'O', 'X');
    EXPECT_LT(score, 0); // AI (O) should detect player can win
}
```

Tool

The testing utilizes the Google Test framework for writing and executing unit tests. This framework provides a structured approach to validate the functionality of the game's individual components, ensuring comprehensive coverage and reliability.

2.2 Integration Testing

Integration testing is essential to validate the interaction and integration of different modules within the Advanced Tic Tac Toe Game. It focuses on ensuring that the components work together seamlessly and that data flows correctly between them. This type of testing is crucial for identifying issues that may not surface during unit testing but can impact the overall functionality and user experience of the game.

Integration tests cover the following components:

1. **UserManager Tests (usermanager_test.cpp)**:
    - **Tested Functions**: registerUser(), loginUser().
    - **Covered Scenarios**: Successfully registering a new user, preventing duplicate usernames, logging in with valid credentials, and blocking login with invalid credentials. These scenarios ensure that the user management system interacts correctly with the users.txt file.
    -

2. **Game History Tests (history_test.cpp)**:
    - **Tested Component**: Writing match records to history.txt.
    - **Covered Scenarios**: Records all moves made in a match, saves both player names and match result, and confirms the file history.txt is correctly created and updated. This ensures proper data flow and persistence for game history.

3. **Leaderboard Tests (leaderboard_test.cpp)**:
    - **Tested Functions**: updateLeaderboard(), showLeaderboard().
    - **Covered Scenarios**: Updates win/loss/draw stats correctly, stores all records in score.txt, and displays sorted ranking based on wins. This validates the integration of game outcomes with the leaderboard system and its file storage.

Tool

Integration tests are performed using the Google Test framework. The interactions with external files like users.txt, score.txt, and history.txt are part of these integration tests.

3.2 Test Execution Screenshots

The following screenshots demonstrate the successful execution of various unit and integration tests using the Google Test framework, providing visual confirmation of the tested functionalities.

Figure 3.2.1: UserManager Tests Execution

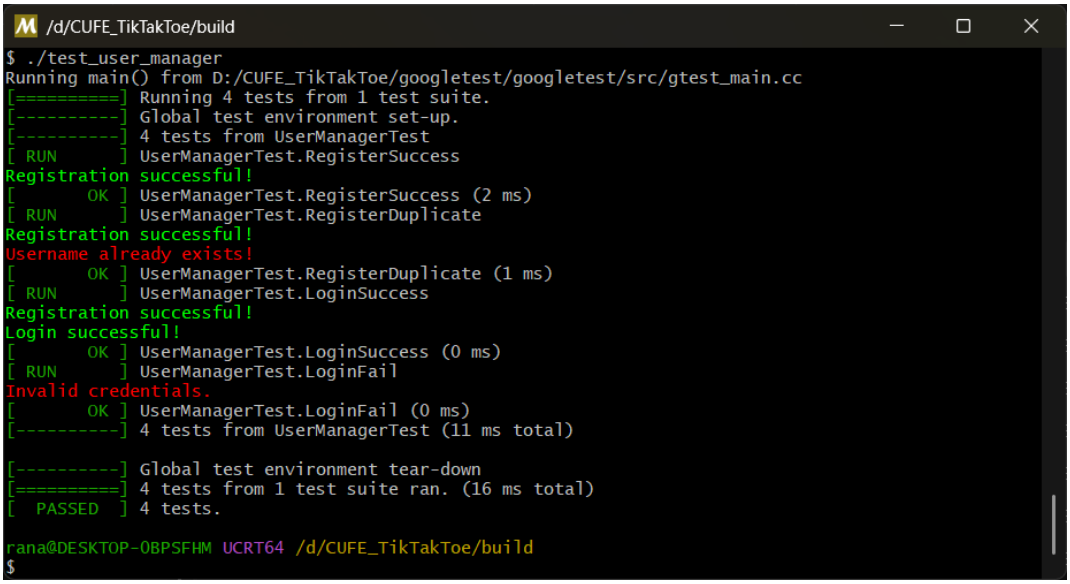


Figure 3.2.2: Game History Tests Execution

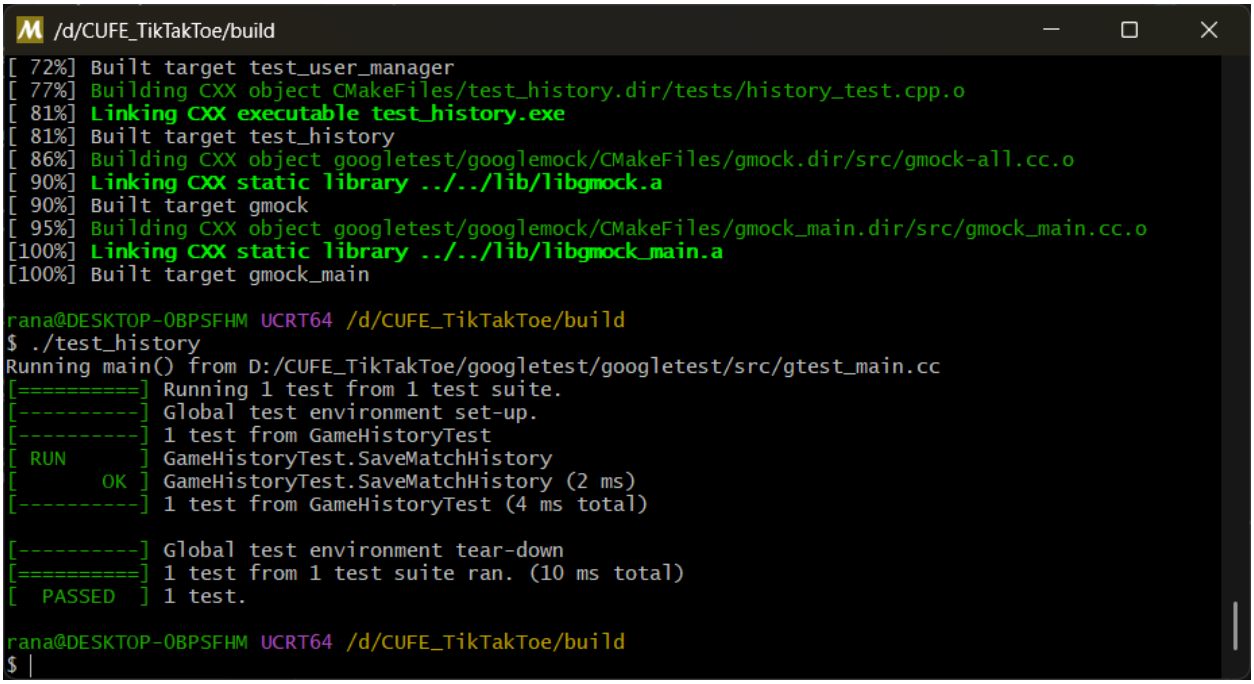


Figure 3.2.3: Leaderboard Tests Execution

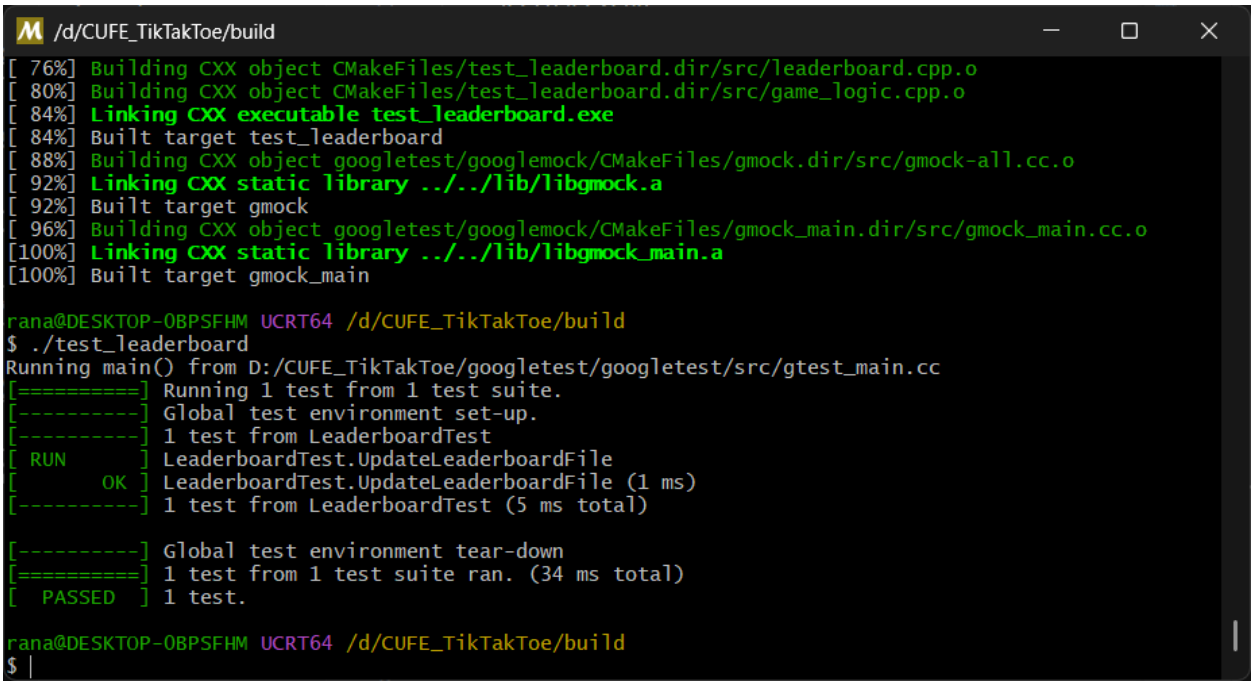


Figure 3.2.4: Easy AI Tests Execution

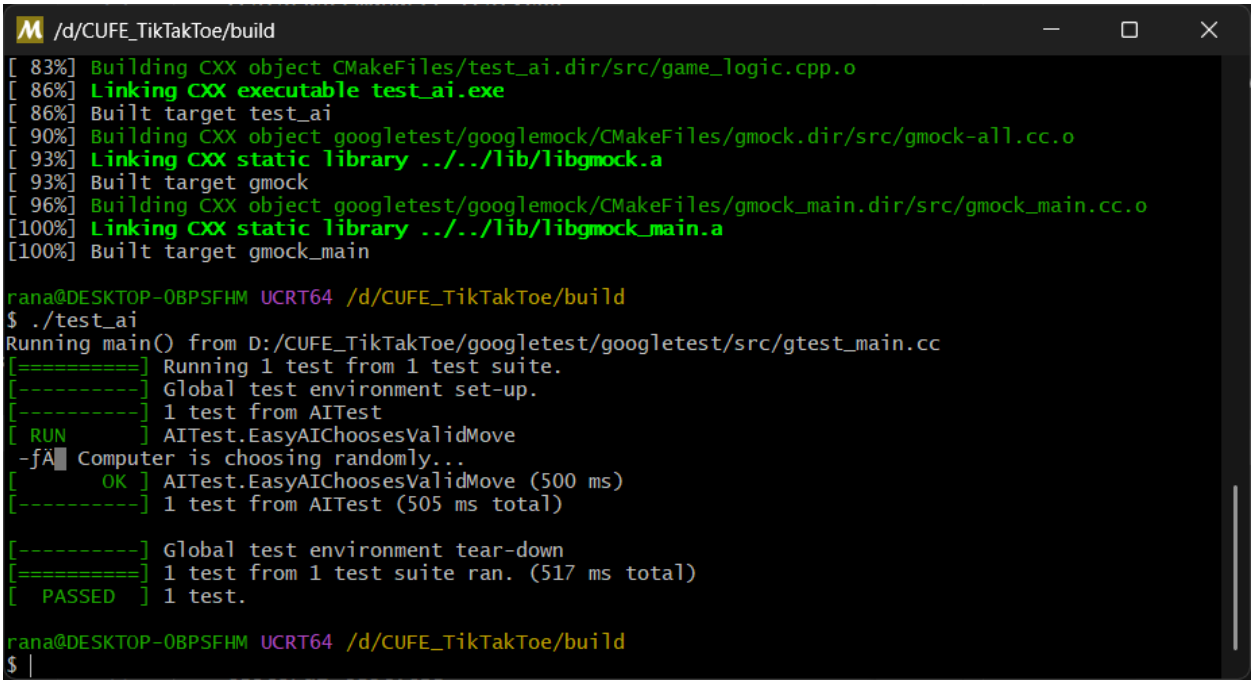


Figure 3.2.5: Medium AI Tests Execution

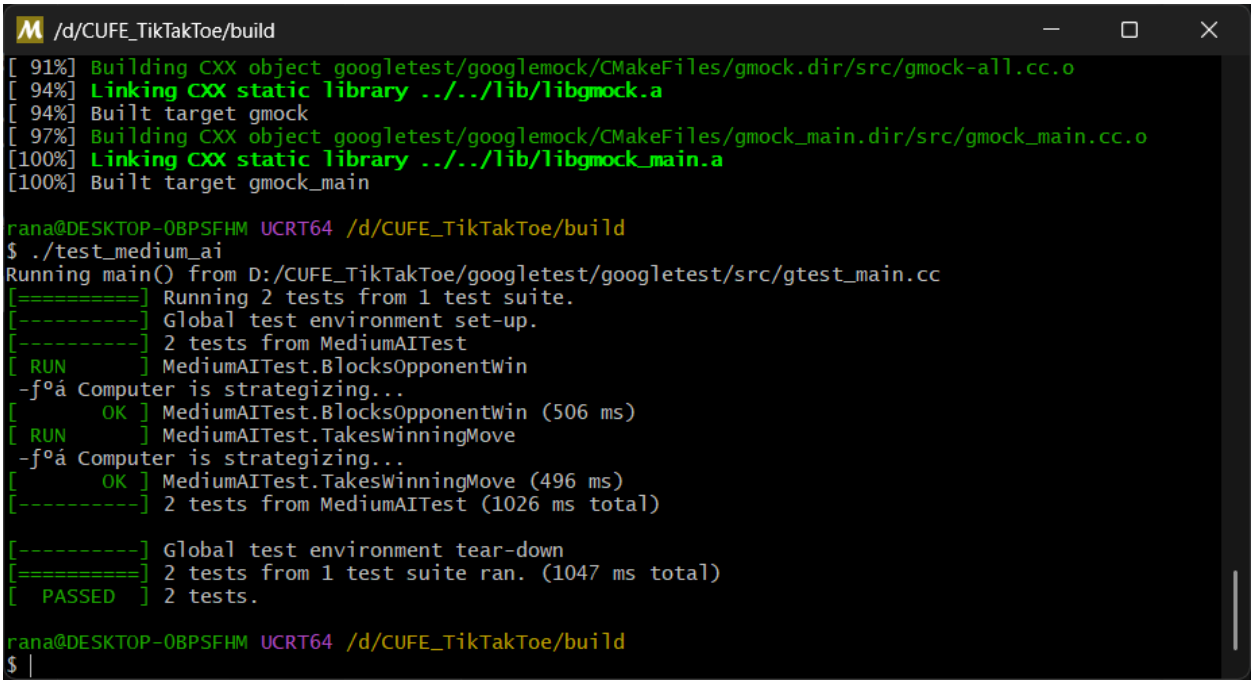


Figure 3.2.6: Hard AI Tests Execution

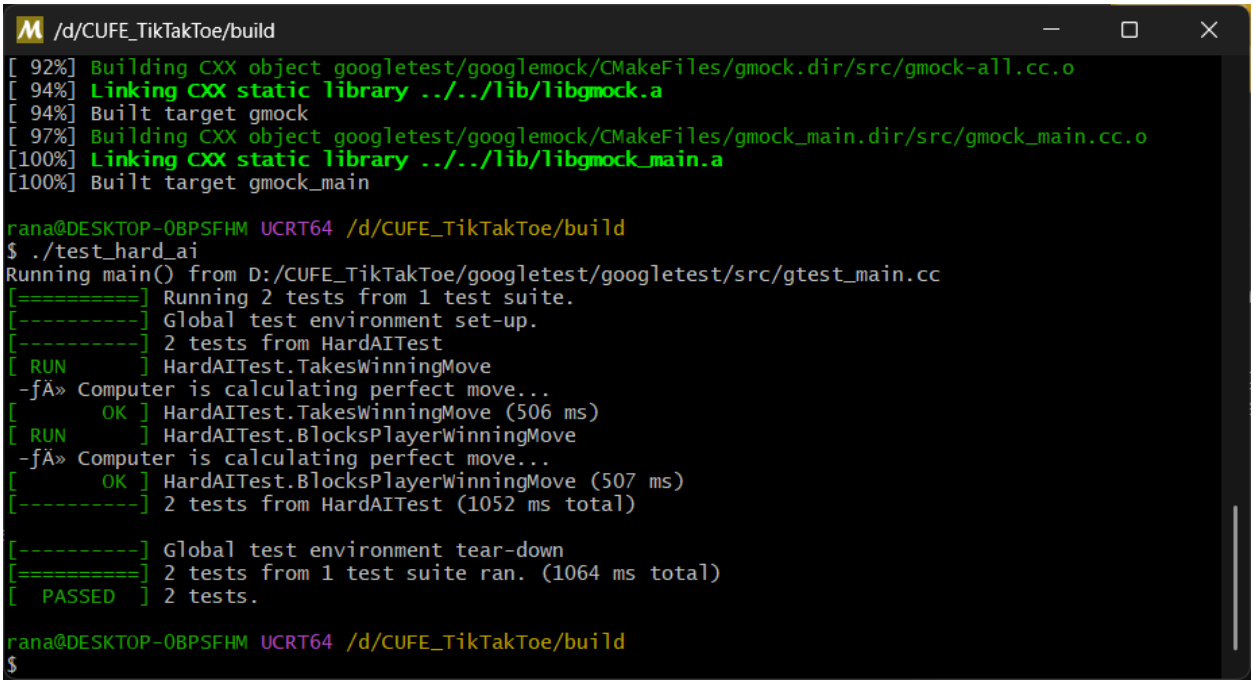
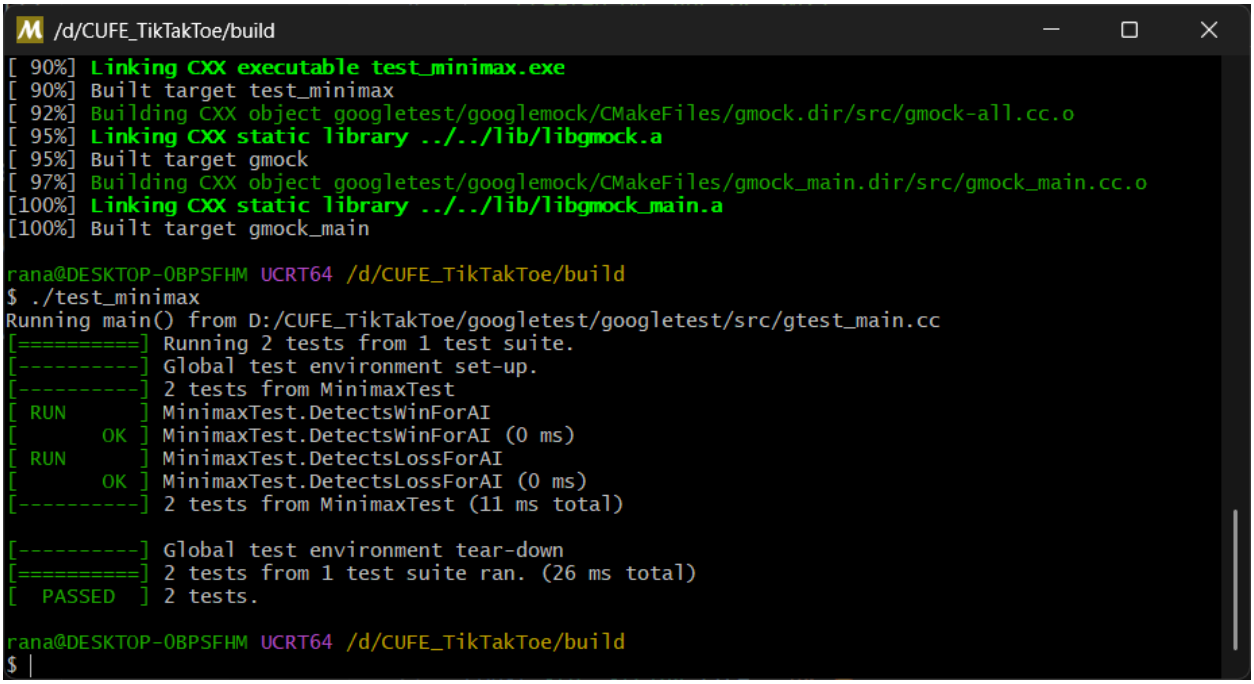


Figure 3.2.7: Minimax Algorithm Tests Execution



4. Conclusion

The testing strategy used in this project effectively identified and resolved potential issues, resulting in a highly reliable and functional Advanced Tic Tac Toe Game. By implementing comprehensive unit and integration testing, along with detailed coverage analysis, all critical aspects of the application, including authentication, game history, leaderboard management, and AI behavior, were thoroughly examined and validated. The use of Google Test and the rigorous testing of all outlined scenarios ensure the robustness and quality of the game.